PHÂN TÍCH THIẾT KẾ THUẬT TOÁN

Chương 1

MỘT SỐ KHÁI NIỆM CƠ BẢN

GV: Bùi Văn Thượng
Email: thuong.bui@ut.edu.vn

NỘI DUNG CHÍNH

Thuật toán

Biểu diễn thuật toán

Độ phức tạp của thuật toán

Cấu trúc dữ liệu

Các chiến lược thiết kế thuật toán

THUẬT TOÁN ALGORITHM

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Thuật toán là một tập hợp hữu hạn các chỉ dẫn rõ ràng, được sắp xếp theo một trình tự logic để giải quyết một vấn đề hoặc thực hiện một nhiệm vụ cụ thể.

Một thuật toán chuẩn phải đáp ứng 5 tiêu chí:

- Đầu vào (Input): Có dữ liệu để xử lý (có thể là không).
- Đầu ra (Output): Phải trả về ít nhất một kết quả.
- Tính xác định: Mỗi bước phải rõ ràng, không gây hiểu lầm.
- Tính hữu hạn: Thuật toán phải dừng lại sau một số bước nhất định (không chạy mãi mãi).
- Tính hiệu quả: Các bước phải đủ đơn giản để có thể thực hiện được.



THUẬT TOÁN ALGORITHM

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

VÍ DỤ: Thuật toán Euclid là thuật toán tìm ước số chung lớn nhất (USCLN) của hai số nguyên dương a và b.

- Input: a, b là số nguyên dương.
- Output: USCLN của a và b.

Thuật toán tìm Euclid có thể được mô tả như sau:

- Bước 1: Nếu a < b thì hoán vị hai số a, b cho nhau.
- Bước 2: Nếu b = 0 thì USCLN là a.
- Bước 3: Ngược lại a > b, thì thực hiện :
  - Tìm số dư r của phép chia a cho b.
  - Gán a = b, b = r, rồi quay trở lại bước 2.

BIỂU DIỄN THUẬT TOÁN

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

CÁC CÁCH BIỂU DIỄN THUẬT TOÁN

- Dùng ngôn ngữ tự nhiên
- Sử dụng mã giả
- Vẽ lưu đồ thuật toán
- Dùng ngôn ngữ lập trình

BIỂU DIỄN THUẬT TOÁN

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

BIỂU DIỄN THUẬT TOÁN SỬ DỤNG NGÔN NGỮ TỰ NHIÊN

- Ta sử dụng ngôn ngữ con người để liệt liệt kê tuần tự các bước để giải quyết bài toán.
- Cách này đơn giản tuy nhiên nó thường dài dòng và đôi khi khó hiểu do cách diễn đạt và vốn từ của người viết thuật toán.

VÍ DỤ: Thuật toán tính tổng hai số a và b:

- Bước 1 : Nhập vào các số a và b;
- Bước 2 : Tính tổng a+b;
- Bước 3 : Xuất kết quả của tổng a+b.

BIỂU DIỄN THUẬT TOÁN

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

BIỂU DIỄN THUẬT TOÁN SỬ DỤNG MÃ GIẢ

- Mã giả là một ngôn ngữ gần giống với ngôn ngữ lập trình. Nó sử dụng kết hợp ngôn ngữ tự nhiên, các ký hiệu toán học, và vay mượn một số cấu trúc của một ngôn ngữ lập trình nào đó để thể hiện thuật toán.
- Mỗi tác giả viết có mỗi phong cách khác nhau, miễn là trình bày rõ ràng và thể hiện được cách giải quyết bài toán.

VÍ DỤ: Tìm số lớn nhất trong hai số a và b:

Nhập giá trị a,b;
if (a ≥ b) then
Xuất kết quả: Số lớn nhất là a;
else
Xuất kết quả: Số lớn nhất là b;

BIỂU DIỄN THUẬT TOÁN

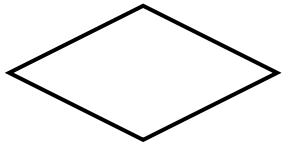
BIỂU DIỄN THUẬT TOÁN BẰNG LƯU ĐỒ

Sử dụng các hình khối khác nhau để biểu diễn giải thuật. Cách này trực quan và dễ hiểu, tuy nhiên hơi cồng kềnh.


Bắt đầu hoặc kết thúc


Input, output


Quá trình xử lý, tính toán

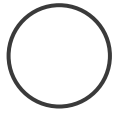

Điều kiện rẽ nhánh


Tệp tin dữ liệu


Khối chương trình con

Hướng đi của thuật toán


Thao tác chuẩn bị

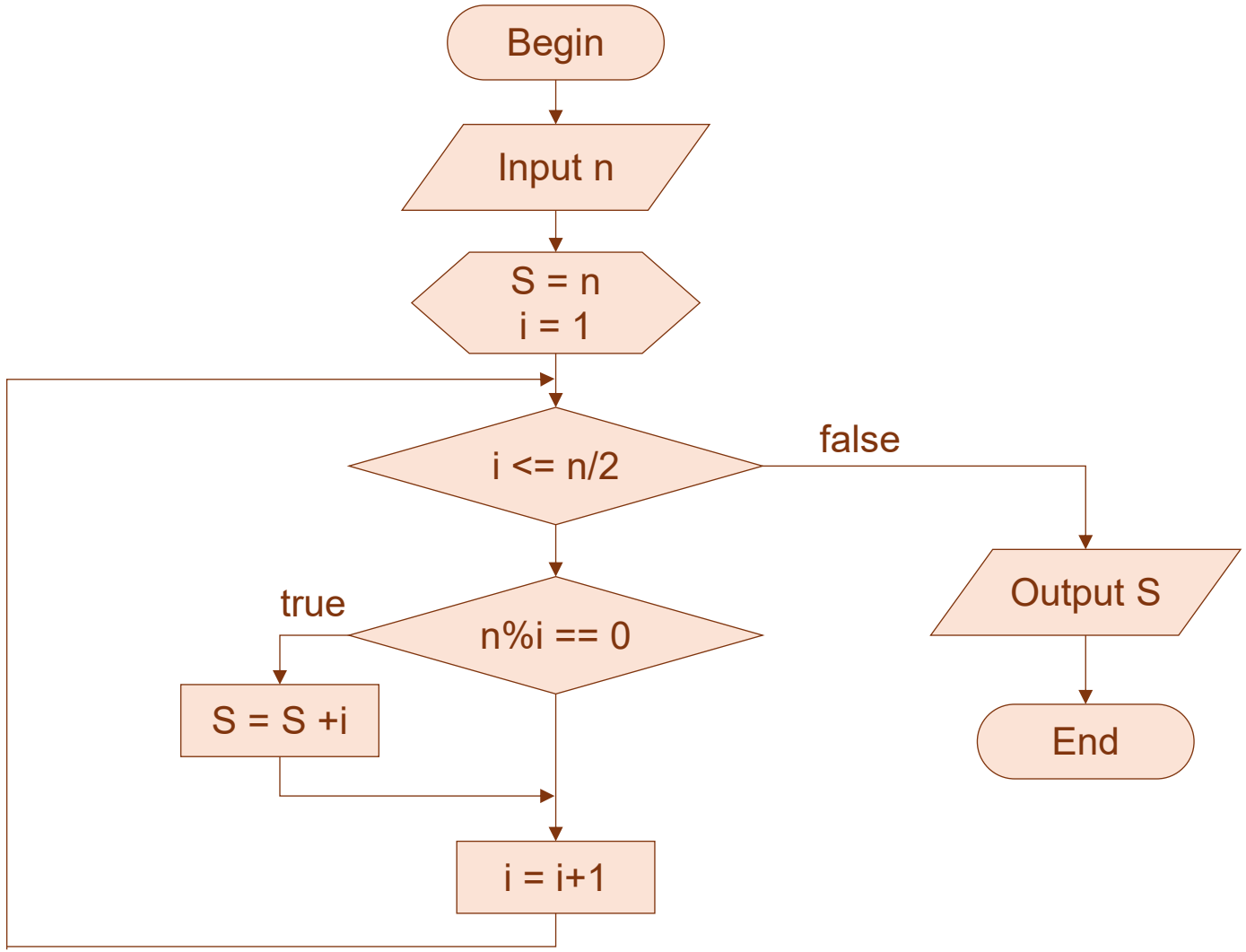

Điểm nối


Chú thích

BIỂU DIỄN THUẬT TOÁN

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

VÍ DỤ: Lưu đồ thuật toán tính tổng các ước của số nguyên dương n.



BIỂU DIỄN THUẬT TOÁN BẰNG NGÔN NGỮ LẬP TRÌNH

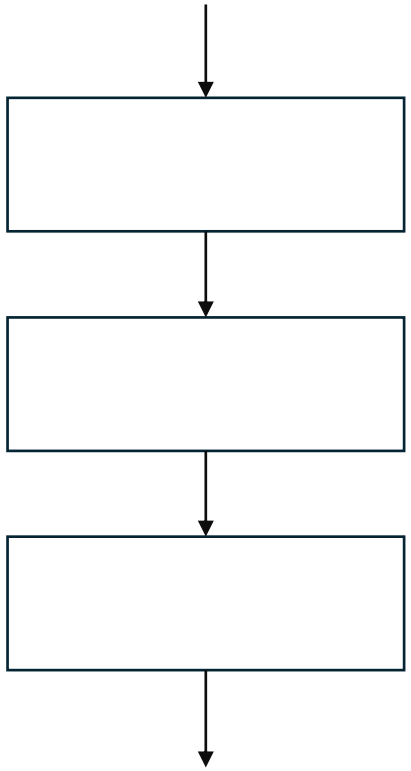
VÍ DỤ: Chương trình c++ cài đặt thuật toán giải phương trình: ax + b = 0

```
#include <iostream>
using namespace std;
int main() {
    double a, b;
    cout << "Nhap he so a: "; cin >> a;
    cout << "Nhap he so b: "; cin >> b;
    if (a != 0) {
        double x = -b / a;
        cout << "Phuong trinh co nghiem duy nhat: x = " << x << endl;
    } else {
        if (b == 0) {
            cout << "Phuong trinh co vo so nghiem." << endl;
        } else {
            cout << "Phuong trinh vo nghiem." << endl;
        }
    }
    return 0;
}
```

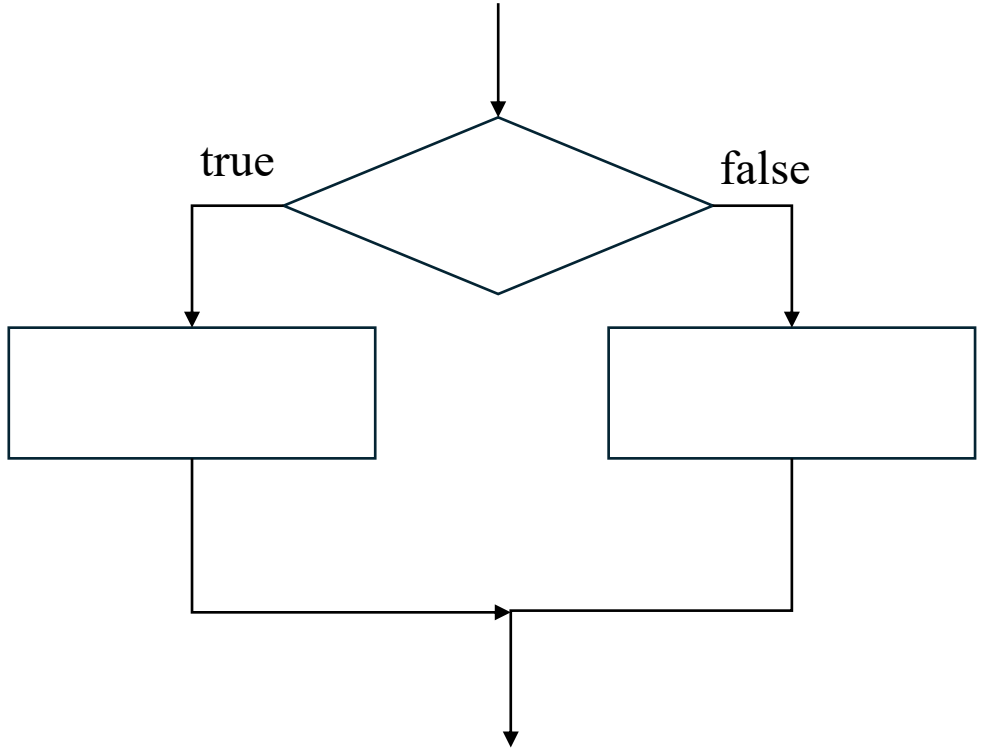
BIỂU DIỄN THUẬT TOÁN

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

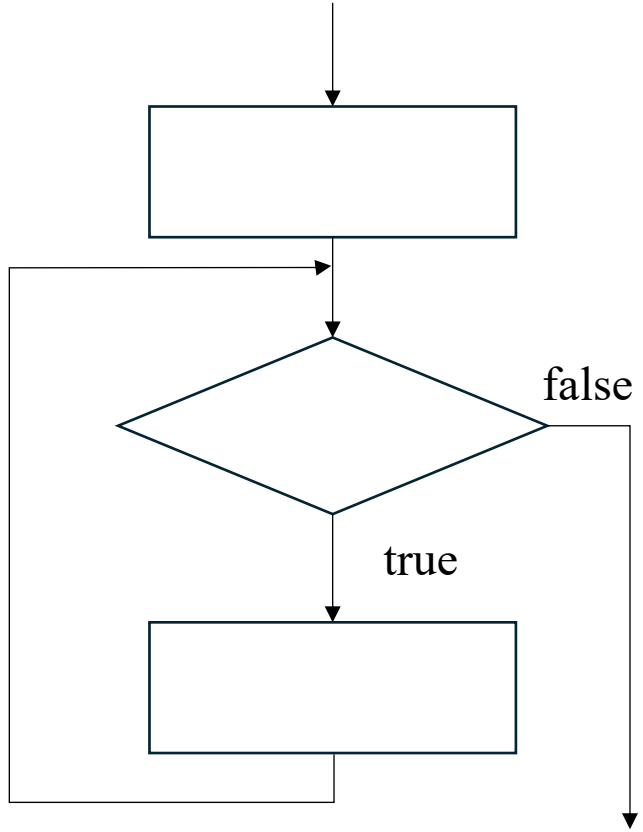
CÁC DẠNG THUẬT TOÁN CƠ BẢN



Cấu trúc tuần tự



Cấu trúc rẽ nhánh



Cấu trúc lặp

VÌ SAO CẦN PHÂN TÍCH ĐỘ PHỨC TẠP CỦA THUẬT TOÁN

Với phần lớn các bài toán, thường có nhiều giải thuật khác nhau để giải. Phân tích độ phức tạp của một giải thuật giúp ta:

- Đánh giá hiệu năng thuật toán.
- So sánh các thuật toán giải cùng một bài toán.
- Dự đoán khả năng chạy trên dữ liệu lớn.
- Tránh phụ thuộc vào phần cứng.

ĐỘ PHỨC TẠP CỦA THUẬT TOÁN

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

KHÁI NIỆM VỀ ĐỘ PHỨC TẠP CỦA THUẬT TOÁN

Độ phức tạp của thuật toán là thước đo lượng tài nguyên mà thuật toán cần để chạy được biểu diễn theo kích thước dữ liệu đầu vào.

Có hai loại chính:

- Độ phức tạp thời gian: số bước thực hiện.
- Độ phức tạp không gian: lượng bộ nhớ sử dụng.

ĐỘ PHỨC TẠP CỦA THUẬT TOÁN

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

ĐỘ PHỨC TẠP THỜI GIAN

- Việc đo thời gian chạy thực tế của thuật toán phụ thuộc nhiều vào tốc độ CPU, ngôn ngữ lập trình, trình biên dịch và các máy tính khác nhau.
- Việc này tốn kém chi phí và thời gian thực nghiệm, vì vậy người ta thường đánh giá thời gian chạy thuật toán bằng phương pháp xấp xỉ toán học thông qua khái niệm Big O.
- Big O dùng để mô tả trường hợp xấu nhất (Worst-case). Nó cho chúng ta biết: Khi dữ liệu tiến tới vô cùng, thời gian chạy của thuật toán sẽ tăng trưởng theo hàm số nào?.

ĐỘ PHỨC TẠP CỦA THUẬT TOÁN

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

CÁC MỨC BIG O PHỔ BIẾN TỪ TỐT ĐẾN XẤU

- O(*1*) hằng số.
- O(log *n*) – logarit.
- O(*n*) tuyến tính.
- O(*n*log *n*) tuyến tính logarit.
- O(n^2) bậc hai.
- O(2^n) hàm mũ.
- O(*n!*) giai thừa.

ĐỘ PHỨC TẠP CỦA THUẬT TOÁN

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

QUY TẮC XÁC ĐỊNH BIG O

- Quy tắc cộng: Nếu thuật toán có 2 phần độc lập chạy nối tiếp nhau O(f(n)) và O(g(n)), thì tổng độ phức tạp là O(max(f(n), g(n))).
- Quy tắc nhân: Nếu có vòng lặp lồng nhau, ta nhân độ phức tạp lại.
- Loại bỏ hằng số: O(2n) hay O(5n + 100) thì vẫn chỉ tính là O(*n*). Trong khoa học thuật toán, chúng ta chỉ quan tâm đến bậc cao nhất.

VÍ DỤ: Độ phức tạp thuật toán sau là $O(n * n) = O(n^2)$.

```
for (int i = 0; i < n; i++) {        // Chạy n lần
    for (int j = 0; j < n; j++) {    // Mỗi lần i chạy, j chạy n lần
        // Phép tính cơ bản
    }
}
// Tổng số phép tính cơ bản là n * n = n^2
// Do đó, độ phức tạp thời gian của đoạn mã này là O(n^2).
```

ĐỘ PHỨC TẠP CỦA THUẬT TOÁN

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

ĐỘ PHỨC TẠP KHÔNG GIAN

Độ phức tạp Không gian (Space Complexity) dùng để đo tài nguyên bộ nhớ cần thiết của thuật toán.

VÍ DỤ:

- Mảng n phần tử → O(*n*)
- Dùng đệ quy → thêm stack → có thể O(*n*)
- Thuật toán chỉ dùng vài biến phụ → O(*1*)

Xu hướng hiện đại: Ngày nay bộ nhớ rẻ hơn thời gian, nên đôi khi chúng ta chấp nhận tốn thêm bộ nhớ để đổi lấy tốc độ (đây gọi là Space-Time Tradeoff).

CẤU TRÚC DỮ LIỆU

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Cấu trúc dữ liệu là một cách tổ chức, quản lý và lưu trữ dữ liệu trong máy tính sao cho chúng ta có thể truy cập và sửa đổi một cách hiệu quả nhất.

Việc lựa chọn cấu trúc dữ liệu ảnh hưởng trực tiếp đến tốc độ thuật toán, mức xử dụng bộ nhớ và khả năng mở rộng của hệ thống.

Cấu trúc dữ liệu phân thành 3 nhóm chính:

- Cấu trúc dữ liệu tuyến tính
- Cấu trúc dữ liệu phi tuyến tính
- Bảng băm

CẤU TRÚC DỮ LIỆU DẠNG TUYẾN TÍNH

Dữ liệu được sắp xếp tuần tự từ trước đến sau:

- Mảng (Array): Tập hợp các phần tử cùng kiểu, nằm liên tiếp trong bộ nhớ.
  - Ưu điểm: Truy cập cực nhanh O(*1*) qua chỉ số.
  - Nhược điểm: Kích thước cố định, chèn/xóa chậm O(*n*).
- Danh sách liên kết (Linked List): Các phần tử (Node) kết nối với nhau qua con trỏ.
  - Ưu điểm: Chèn/xóa linh hoạt.
  - Nhược điểm: Truy cập chậm vì phải duyệt từ đầu.
- Ngăn xếp (Stack): LIFO Vào sau ra trước.
- Hàng đợi (Queue): FIFO Vào trước ra trước.

CẤU TRÚC DỮ LIỆU DẠNG PHI TUYẾN TÍNH

Dữ liệu có mối quan hệ phân cấp hoặc mạng lưới:

- Cây (Tree): Cấu trúc phân cấp (như thư mục máy tính). Quan trọng nhất là Cây tìm kiếm nhị phân (BST) giúp tìm kiếm cực nhanh O(log *n*).
- Đồ thị (Graph): Tập hợp các đỉnh và cạnh (như mạng lưới bạn bè trên Facebook hoặc bản đồ giao thông).

BẢNG BĂM (HASH TABLE)

Bảng băm là một cấu trúc dữ liệu dùng để lưu trữ và truy xuất dữ liệu rất nhanh dựa trên hàm băm để chuyển một key thành một địa chỉ bộ nhớ.

- Hiệu năng: Cho phép Tìm kiếm, Chèn, Xóa với độ phức tạp trung bình là O(*1*).
- Ứng dụng: Lưu trữ mật khẩu, bộ nhớ đệm (Cache), cơ sở dữ liệu tốc độ cao (Redis).

CÁC CHIẾN LƯỢC THIẾT KẾ THUẬT TOÁN

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

GIỚI THIỆU CHUNG

Chiến lược thiết kế thuật toán (algorithm design paradigms) là những phương pháp tư duy có hệ thống nhằm xây dựng thuật toán hiệu quả cho các bài toán tính toán. Mỗi chiến lược phản ánh một cách tiếp cận khác nhau đối với việc phân tích cấu trúc bài toán và tổ chức lời giải.

Chiến lược thiết kế thuật toán giúp:

- Lựa chọn phương pháp giải tối ưu.
- Tạo nền tảng cho việc phát triển các thuật toán phức tạp.
- Hỗ trợ đánh giá và so sánh các thuật toán.

CÁC CHIẾN LƯỢC THIẾT KẾ THUẬT TOÁN PHỔ BIẾN

- Chia để trị (Divide and Conquer)
- Quy hoạch động (Dynamic Programming)
- Tham lam (Greedy Method)
- Quay lui (Backtracking)
- Nhánh và cận (Branch and Bound)
- Lập trình tuyến tính (Linear Programming)
- Heuristic và Metaheuristic
- Tìm kiếm và duyệt (Search Strategies)

CHIẾN LƯỢC CHIA ĐỂ TRỊ (DIVIDE AND CONQUER)

Đây là phương pháp luận mang tính nền tảng nhất trong thiết kế thuật toán, dựa trên nguyên lý đệ quy để phân rã bài toán.

- Cơ chế của chiến lược:
  - Chia (Divide): Phân chia bài toán lớn thành các bài toán con cùng loại.
  - Trị (Conquer): Giải quyết các bài toán con bằng đệ quy. Nếu bài toán con đủ nhỏ, giải trực tiếp (trường hợp cơ sở).
  - Kết hợp (Combine): Tổng hợp lời giải của các bài toán con để có lời giải cho bài toán ban đầu.
- Độ phức tạp điển hình: Thường đạt mức O(n log n).
- Ứng dụng tiêu biểu: Sắp xếp trộn (Merge Sort), Sắp xếp nhanh (Quick Sort), Nhân ma trận Strassen.

CHIẾN LƯỢC QUY HOẠCH ĐỘNG (DYNAMIC PROGRAMMING)

Quy hoạch động áp dụng cho bài toán có tính chất con chồng lặp và cấu trúc con tối ưu. Lưu trữ kết quả trung gian để tránh tính toán lặp lại.

- Nguyên lý tối ưu: Lời giải tối ưu cho bài toán lớn được xây dựng từ lời giải tối ưu của các bài toán con.
- Cơ chế lưu trữ: Sử dụng kỹ thuật Ghi nhớ hoặc Lập bảng để tránh tính toán lại các bài toán con đã giải.
- Ứng dụng tiêu biểu:
  - Bài toán Cái túi (Knapsack Problem).
  - Tìm chuỗi con chung dài nhất (LCS).
  - Tìm đường đi ngắn nhất (Floyd-Warshall).

CHIẾN LƯỢC THAM LAM (GREEDY STRATEGY)

Khác với Quy hoạch động luôn xem xét toàn cục, chiến lược Tham lam tập trung vào việc đưa ra lựa chọn tối ưu cục bộ tại mỗi bước với hy vọng dẫn đến tối ưu toàn cục.

- Đặc điểm: Không bao giờ thay đổi quyết định đã đưa ra.
- Ưu điểm: Cài đặt đơn giản, hiệu suất thời gian cực cao.
- Thách thức: Cần chứng minh toán học rằng lựa chọn tham lam sẽ dẫn đến kết quả tối ưu (thường dùng phương pháp trao đổi hoặc quy nạp).
- Ứng dụng tiêu biểu:
  - Thuật toán Prim, Kruskal (Cây khung tối tiểu)
  - Mã hóa Huffman.
  - Thuật toán Dijkstra.

CHIẾN LƯỢC QUAY LUI (BACKTRACKING)

Đây là một dạng cải tiến của phương pháp Vét cạn (Brute Force), thực hiện tìm kiếm lời giải trong không gian trạng thái một cách có hệ thống.

- Cơ chế: Thử từng khả năng theo cấu trúc cây. Nếu phát hiện một nhánh không thể dẫn đến lời giải (nhờ hàm đánh giá hoặc các điều kiện ràng buộc), thuật toán sẽ "quay lui" về bước trước đó để thử nhánh khác.
- Kỹ thuật quan trọng: Cắt tỉa (Pruning) để giảm thiểu không gian tìm kiếm, giúp thuật toán thoát khỏi các nhánh vô vọng sớm nhất có thể.
- Ứng dụng tiêu biểu:
  - Bài toán Tám quân hậu (8-Queens).
  - Giải Sudoku.
  - Bài toán Người đi giao hàng (phiên bản vét cạn).

CHIẾN LƯỢC NHÁNH VÀ CẬN (BRANCH AND BOUND)

Tương tự như Quay lui, nhưng thường được áp dụng cho các bài toán tối ưu hóa (tìm giá trị lớn nhất hoặc nhỏ nhất).

- Cơ chế: Sử dụng các giá trị biên (Bounds) để loại bỏ các vùng không chứa lời giải tối ưu hơn lời giải hiện tại.
- Ứng dụng tiêu biểu:
  - Tối ưu hóa trong sản xuất.
  - Lập lịch trình.
  - Bài toán TSP quy mô vừa.

CHIẾN LƯỢC LẬP TRÌNH TUYẾN TÍNH (LINEAR PROGRAMMING - LP)

Đây là một phương pháp toán học nhằm tối ưu hóa (tìm giá trị cực đại hoặc cực tiểu) của một hàm mục tiêu tuyến tính, với các điều kiện ràng buộc cũng là các bất đẳng thức hoặc phương trình tuyến tính.

- Đặc điểm: Dựa trên mô hình toán học thuần túy. Không gian nghiệm khả thi thường là một đa diện lồi.
- Cơ chế: Sử dụng các thuật toán như Simplex hoặc Interior Point để di chuyển dọc theo các biên của vùng khả thi nhằm tìm ra điểm tối ưu.
- Ứng dụng: Tối ưu hóa danh mục đầu tư, phân bổ nguồn lực sản xuất, quy hoạch mạng lưới giao thông.

CHIẾN LƯỢC HEURISTIC VÀ METAHEURISTIC

Trong khoa học máy tính, Heuristic là một phương pháp tiếp cận "đường tắt", chấp nhận một kết quả "đủ tốt" thay vì hoàn hảo để đổi lấy tốc độ xử lý.

Metaheuristic là một tầng cao hơn của Heuristic. Đây là các khuôn mẫu thuật toán trừu tượng được thiết kế để thoát khỏi các "bẫy" tối ưu cục bộ (local optima) nhằm tìm kiếm lời giải gần tối ưu trên quy mô toàn cục.

Ứng dụng:

- Trò chơi điện tử (AI pathfinding)
- Nhận dạng mẫu.
- Các bài toán xếp lịch đơn giản.
- Giải các bài toán NP-Hard như Người đi giao hàng (TSP) quy mô lớn.
- tối ưu hóa cấu trúc protein.

CÁC CHIẾN LƯỢC THIẾT KẾ THUẬT TOÁN

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

CHIẾN LƯỢC TÌM KIẾM VÀ DUYỆT (SEARCH & TRAVERSAL)

Đây là chiến lược nền tảng để khám phá không gian trạng thái của bài toán, thường được biểu diễn dưới dạng đồ thị hoặc cây.

Tìm kiếm mù (Uninformed Search): Không có thông tin về mục tiêu.

- Duyệt theo chiều rộng (BFS.
- Duyệt theo chiều sâu (DFS.

Tìm kiếm có thông tin (Informed Search):

- Best-First Search.
- A* Search.

Ứng dụng: Trí tuệ nhân tạo, đồ thị, mạng máy tính, tối ưu hóa đường đi.

THAM KHẢO THÊM

[1] Hướng dẫn vẽ lưu đồ thuật toán: https://youtu.be/t_E3BlhTU2o

BÀI TẬP THỰC HÀNH CHƯƠNG 1

Bài 1. Cho một mảng A gồm n số nguyên. Vẽ lưu đồ tìm giá trị lớn nhất (Max) của mảng A và xác định độ phức tạp Big O của thuật toán.

Bài 2. Vẽ lưu đồ cho thuật toán Sắp xếp nổi bọt (Bubble Sort) để sắp xếp mảng tăng dần. Tính toán Big O trong trường hợp xấu nhất.

Bài 3. Vẽ lưu đồ cho thuật toán Tìm kiếm nhị phân (Binary Search) trên một mảng đã sắp xếp. Tính toán Big O trong trường hợp xấu nhất.

Bài 4. Vẽ lưu đồ mô tả thuật toán kiểm tra số nguyên tố. Tính toán Big O.

Bài 5. Tính số lần lặp và tính toán Big O của vòng lặp sau:

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++) {
        cout<< i << " " << j << endl;
    }
}
```

THANKS